F **Farmloop**
Build log & test guide

FARM MANAGEMENT SYSTEM — BUILD CHECKPOINT

Farmloop, documented so far

An integrated farm management app covering animals, crops, fish, and fungi operations — a Django REST Framework backend and a Flutter mobile client, built one feature at a time and verified against a real database and a real iOS Simulator, not just a screenshot. This document is the checkpoint before manual product testing begins.

`Django 6.1 · DRF` `PostgreSQL` `Celery + Beat` `Flutter · Riverpod 3` `go_router` `JWT auth` `Cloudinary · Stripe`

8	4	6/7	2
Backend apps	Mobile tabs shipped	Team actions verified	Features scaffolded, not built

01 Backend

Django REST Framework, 8 apps, farm-scoped everything. Auth is JWT with social login plus a DEBUG-only dev login for testing without real OAuth credentials.

Apps & models

APP	MODELS	PURPOSE
core	UUIDModel, TimestampedModel, ArchivableModel	Shared abstract bases. Archive-not-delete via `ArchivableModel`.
accounts	User	Custom user (UUID PK), role + farm membership, social auth fields.
farms	Farm, Zone	A farm and its physical zones (coop, plot, storage…).
inventory	LivingInventoryItem, AnimalDetail, CropDetail, FishDetail, FungiDetail, HealthLogEntry, ProductionLogEntry, BreedingLogEntry	Base item + domain-specific detail row, plus append-only logs.
supplies	Supply, SupplyConsumptionGroup, ConsumptionLogEntry, SupplyForecast	Stock tracking, linkage to consuming inventory, consumption history, depletion forecasts.

APP	MODELS	PURPOSE
schedules	Schedule, Occurrence	Recurring rule vs. individual task instance. Occurrence denormalizes title/urgency for offline rendering.
audit	AuditLogEntry	Immutable trail: actor, action, category, target, description.
notifications	DeviceToken, Notification	Push token registration + in-app notifications.
billing	SubscriptionEvent	Stripe subscription lifecycle events.

User.Role

ADMIN · FARMHAND

Farm.PlanTier

FREE · TIER_1 · TIER_2 · TIER_3

LivingInventoryItem.DomainType

ANIMAL · CROP · FISH · FUNGI

Schedule / Occurrence urgency

URGENT · ROUTINE — occurrences also carry OVERDUE

API surface — /api/v1/

ENDPOINT	NOTES
POST auth/social/google/, auth/social/apple/	Social login; invite-then-claim account matching.
POST auth/dev-login/	**DEBUG-only.** Mints tokens for an existing user by email — no real OAuth needed.
POST auth/token/refresh/	JWT refresh.
GET/PATCH me/	Current user profile.
GET farm/, farm/modules/	Farm detail and enabled modules (animals/plants/fish/fungi).
/zones/	Farm zones CRUD (ViewSet).
GET team/, POST team/invite/, PATCH/DELETE team/<id>/	Team mgmt, Admin-only. DELETE detaches (`farm=None, role=None`), doesn't delete the account.
/inventory/ + .../health-log/, production-log/, breeding-log/	Living inventory CRUD + per-item logs.
/supplies/	Supply CRUD, stock, consumption (ViewSet).
/schedules/, /occurrences/	Recurring rules + task instances — powers the Triage feed.
GET audit/	Farm-scoped audit trail.
notifications/register-device/, notifications/, notifications/<id>/read/	Push tokens + in-app notifications.
billing/checkout-link/, billing/webhook/stripe/	Stripe checkout + webhook.
/api/schema/, /api/docs/	OpenAPI schema + Swagger UI.

Permissions

`apps/core/permissions.py` — `IsFarmMember` (any authenticated user with a farm), `IsFarmAdmin` (role must be ADMIN; gates team/audit/analytics/schedule editing), and `require_plan_tier(*tiers)`, a factory gating a view to specific subscription tiers independent of role.

Background jobs — Celery Beat

TASK	SCHEDULE	PURPOSE
`generate_upcoming_occurrences`	Daily, 01:00 UTC	Generates Occurrences from active Schedules, 21-day horizon. Idempotent — self-heals a missed run.
`recompute_supply_forecasts`	Daily, 02:00 UTC	Recomputes depletion forecasts from consumption history.

Management commands

- `manage.py seed_demo` — seeds a realistic demo farm, idempotent (owner + farmhand, 1 farm, 3 zones, 3 inventory items, 2 supplies, 3 recurring schedules with generated occurrences, 2 audit entries).
- `manage.py demo_token [email]` — prints a fresh JWT access token for Swagger's Authorize button. Omit the email for both demo users at once.

02 Mobile app

Flutter + Riverpod 3, feature folders mirroring the backend apps. Bottom-nav shell built on `StatefulShellRoute.indexedStack`.

Navigation

TAB	ROUTE	SCREEN
Triage	`/`	`HomeScreen` — today's due/overdue occurrences, swipe-to-complete.
Inventory	`/inventory`, `/inventory/:id`	`LivingInventoryScreen`, `InventoryDetailScreen` — tabbed by Animals/Plants/Fish/Fungi.
Supplies	`/supplies`, `/supplies/:id`	`SuppliesScreen`, `SupplyDetailScreen`.
Team ● Admin only	`/team`	`TeamAuditScreen` — Members + Activity tabs.

> The Team tab is hidden from the nav bar for Farmhands *and* the router redirects away from `/team` for a non-admin deep link — defense in depth matching the backend's own `IsFarmAdmin` gate.

Login has two **Dev Login** buttons (DEBUG builds only) hitting the backend's `dev-login` endpoint directly:

- **Dev Login — Owner (Admin)** → `owner@farmloop.demo`
- **Dev Login — Sam (Farmhand)** → `sam@farmloop.demo`

Feature status

FEATURE	STATUS
Auth (login, dev-login, token storage/refresh)	Verified
Triage / Home (occurrence feed, swipe-to-complete)	Verified
Living Inventory (list, detail, domain tabs)	Verified
Supplies (list, detail, stock)	Verified
Team — promote / demote / remove	Verified
Team — invite dialog	Needs manual test
Audit trail (Activity tab)	Verified
`farm/` and `notifications/` feature folders	Scaffolded only — no UI

State management notes

- `AsyncValue.valueOrNull` doesn't exist in Riverpod 3.x — use `.value` directly (nullable in this version).
- An action-only controller read solely via `.notifier` (never `.watch`'d) has nothing keeping it alive under default `autoDispose`, and can be disposed mid-mutation, silently dropping the `ref.invalidate()` that should refresh the UI. Every action controller here (`TeamActionsController`, `SupplyActionsController`, …) is marked `@Riverpod(keepAlive: true)` for exactly this reason.

Dev workflow quirks

- **Web preview:** `flutter run -d web-server` hangs in this sandbox (no Dart Debug Chrome Extension). Use `flutter build web --debug -o build/web` then serve with a static file server — wired into `.claude/launch.json`.
- **iOS Simulator:** tap coordinates are in device points, not screenshot pixels (iPhone 17 Pro: 402×874pt, screenshot-px→pt scale ≈2.3). There's real latency between a tap and the screenshot reflecting it — a screenshot alone isn't proof an action registered; mutating actions were re-verified directly against the backend.

03 Running it locally

Backend needs PostgreSQL and Redis running locally.

Backend

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env   # fill in DATABASE_URL at minimum
python manage.py migrate
python manage.py seed_demo
python manage.py runserver 0.0.0.0:8000
```

Background jobs: `celery -A config worker -l info` and `celery -A config beat -l info`.

Mobile — web preview

```
cd mobile
flutter build web --debug -o build/web
cd build/web && python3 -m http.server 5050
```

Mobile — iOS Simulator

```
cd mobile
flutter build ios --debug --simulator
```

Install the built `.app` onto a booted simulator.

Demo accounts

ROLE	EMAIL	NOTES
Admin (owner)	`owner@farmloop.demo`	Jane Farmer — sees the Team tab.
Farmhand	`sam@farmloop.demo`	Sam Rivers — no Team tab.

Sign in with the **Dev Login** buttons — no real Google/Apple credentials needed.

04 Status & open items

> **Invite dialog (Team screen)** — not verified via simulator automation. The FAB tap didn't register across 7 calibrated attempts, while every other tap on the same screen worked with the same calibration. Code review shows a standard `showDialog`, consistent with the rest of the app — reads as an automation-tool limitation, not an app bug, matching an earlier confirmed case with the Triage swipe gesture. **Needs a manual tap to confirm.**

- `farm/` and `notifications/` mobile features are scaffolded folders only — no screens built yet.
- No automated integration/E2E suite beyond `flutter analyze` / `flutter test` (unit-level) and manual backend-query verification during simulator testing.
- The project is not yet under git version control.

05 Manual testing checklist

For testing on your own device now that work is paused.

Auth

- [ ] Dev Login as Owner — lands on Triage, Team tab visible
  owner@farmloop.demo
- [ ] Dev Login as Sam — lands on Triage, Team tab hidden
  sam@farmloop.demo
- [ ] App relaunch keeps the session (token persists)

Triage

- [ ] Due/overdue occurrences list correctly for today
- [ ] Swipe-to-complete works by hand, on device
  Known automation-tool limitation — this needs a real finger
- [ ] Completing a supply-linked task deducts stock correctly

Inventory

- [ ] All four domain tabs load — Animals / Plants / Fish / Fungi
- [ ] Item detail screen shows correct domain-specific fields and logs

Supplies

- [ ] Supply list shows current stock and reorder status
- [ ] Supply detail shows consumption history

Team & Audit `Admin only`

- [ ] Promote a farmhand to Admin, then demote back
- [ ] Remove a member from the farm
- [ ] **Invite dialog:** tap Invite, submit an email, confirm the new member appears
  Unverified by automation — priority check
- [ ] Activity tab reflects the actions above with correct actor names and timestamps
- [ ] Farmhand account cannot reach /team via deep link or nav

Cross-cutting

- [ ] Pull-to-refresh works on each list screen
- [ ] Back navigation from every detail screen returns to the right list state
- [ ] No crashes switching rapidly between all four tabs
